

OS作业7-郭威威-2414308

生产者消费者问题实验报告

一、实验目的

采用多线程结合Posix信号量与互斥量，实现I个生产者、J个消费者的同步互斥问题，满足共享缓冲区大小为N、总生产消费K ($K > N$) 个产品的核心需求，同时完成指定行为输出、边界处理等要求。

二、实验环境

- 操作系统：Ubuntu（基于Linux内核，支持Posix接口）
- 编译器：GCC（通过build-essential套件安装）
- IDE（可选）：Code::Blocks
- 核心库：pthread（线程库）、semaphore（信号量库）、time（时间库）

三、实验原理

1. 核心技术选型

- **Posix信号量**：用于同步控制，empty信号量（初始值N）控制空缓冲区数量，full信号量（初始值0）控制满缓冲区数量。
- **Posix互斥量**：用于保护临界区（缓冲区操作），避免多线程并发访问导致的数据不一致。
- **多线程**：通过pthread库创建生产者线程与消费者线程，实现并发执行。

2. 同步互斥逻辑

- 生产者流程：P(empty) → P(mutex) → 临界区操作（生产产品→存入缓冲区）→ V(mutex) → V(full)
- 消费者流程：P(full) → P(mutex) → 临界区操作（取出产品→消费产品）→ V(mutex) → V(empty)
- 边界控制：通过全局计数器记录已生产/已消费产品数量，达到K时终止对应线程。

四、实验步骤

1. 环境搭建

1. 安装编译依赖：执行 `sudo apt-get install build-essential` 安装GCC及相关工具。

2. (可选) 安装IDE: 执行 `sudo apt-get install codeblocks` , 后续可通过IDE编写调试代码。

2. 代码设计与编写

(1) 头文件与宏定义

代码块

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <time.h>
5  #include <stdlib.h>
6  #include <stdbool.h>
7  #include <stdint.h>
8  #include <unistd.h>
9
10 #define DATA_SIZE 256    // 产品数据大小
11 #define N 5              // 缓冲区大小 (可修改)
12 #define I 2              // 生产者数量 (可修改)
13 #define J 3              // 消费者数量 (可修改)
14 #define K 20             // 总产品数量 (可修改, 需满足K>N)
```

(2) 结构体定义

代码块

```
1  // 产品结构体
2  typedef struct item_st {
3      int product_id;        // 产品编号 (1~K)
4      time_t timestamp_p;    // 生产时间戳
5      int producer_id;       // 生产者编号 (1~I)
6      uint32_t checksum;     // 产品校验码 (简单计算数据总和)
7      uint32_t data[DATA_SIZE]; // 产品详细数据
8  } ITEM_ST;
9
10 // 缓冲区节点结构体
11 typedef struct buffer_st {
12     int buffer_id;          // 缓冲区编号 (1~N)
13     bool isempty;           // 是否为空标记
14     ITEM_ST item;           // 存储的产品
15     struct buffer_st* nextbuf; // 链表指针 (实现循环缓冲区)
16 } BUFFER_ST;
```

(3) 全局变量声明

代码块

```
1  BUFFER_ST buffer[N];      // 共享缓冲区数组
2  sem_t empty, full;        // 同步信号量
3  pthread_mutex_t mutex;    // 互斥量
4  int produced_cnt = 0;      // 已生产产品计数
5  int consumed_cnt = 0;      // 已消费产品计数
6  pthread_mutex_t cnt_mutex; // 保护计数的互斥量
```

(4) 工具函数实现

代码块

```
1  // 计算产品校验码 (数据数组总和)
2  uint32_t calc_checksum(uint32_t data[]) {
3      uint32_t sum = 0;
4      for (int i = 0; i < DATA_SIZE; i++) sum += data[i];
5      return sum;
6  }
7
8  // 初始化缓冲区 (循环链表)
9  void init_buffer() {
10     for (int i = 0; i < N; i++) {
11         buffer[i].buffer_id = i + 1;
12         buffer[i].isempty = true;
13         buffer[i].nextbuf = &buffer[(i + 1) % N]; // 循环链接
14     }
15 }
```

(5) 生产者线程函数

代码块

```
1  void* producer_thread(void* arg) {
2      int producer_id = *(int*)arg; // 生产者编号
3      free(arg); // 释放动态分配的编号内存
4  }
```

```
5     while (1) {
6         // 生产随机等待时间 (100~500ms)
7         int sleep_ms = rand() % 401 + 100;
8         usleep(sleep_ms * 1000);
9
10        // 检查是否已生产完K个产品, 避免超产
11        pthread_mutex_lock(&cnt_mutex);
12        if (produced_cnt >= K) {
13            pthread_mutex_unlock(&cnt_mutex);
14            printf("生产者%d: 已完成%d个产品生产, 退出线程\n", producer_id, K);
15            pthread_exit(NULL);
16        }
17        pthread_mutex_unlock(&cnt_mutex);
18
19        // P操作: 申请空缓冲区
20        sem_wait(&empty);
21        // P操作: 申请临界区访问权
22        pthread_mutex_lock(&mutex);
23
24        // 查找空缓冲区
25        BUFFER_ST* empty_buf = &buffer[0];
26        while (!empty_buf->isempty) empty_buf = empty_buf->nextbuf;
27
28        // 生产产品
29        produced_cnt++;
30        ITEM_ST new_item;
31        new_item.product_id = produced_cnt;
32        new_item.timestamp_p = time(NULL);
33        new_item.producer_id = producer_id;
34        // 填充随机数据
35        for (int i = 0; i < DATA_SIZE; i++) new_item.data[i] = rand() % 1000;
36        new_item.checksum = calc_checksum(new_item.data);
37
38        // 存入缓冲区
39        empty_buf->item = new_item;
40        empty_buf->isempty = false;
41
42        // 输出行为信息
43        printf("生产者%d: 准备进入临界区\n", producer_id);
44        printf("生产者%d: 已进入临界区\n", producer_id);
45        printf("生产者%d: 生产产品%d (校验码: %u, 生产时间: %ld) \n",
46            producer_id, new_item.product_id, new_item.checksum,
47            new_item.timestamp_p);
48        printf("生产者%d: 将产品%d存入缓冲区%d\n",
49            producer_id, new_item.product_id, empty_buf->buffer_id);
50        printf("生产者%d: 已离开临界区\n", producer_id);
```

```

51         // V操作: 释放临界区
52         pthread_mutex_unlock(&mutex);
53         // V操作: 增加满缓冲区计数
54         sem_post(&full);
55     }
56 }

```

(6) 消费者线程函数

代码块

```

1  void* consumer_thread(void* arg) {
2      int consumer_id = *(int*)arg; // 消费者编号
3      free(arg); // 释放动态分配的编号内存
4
5      while (1) {
6          // 消费随机等待时间 (150~600ms)
7          int sleep_ms = rand() % 451 + 150;
8          usleep(sleep_ms * 1000);
9
10         // 检查是否已消费完K个产品, 避免超消费
11         pthread_mutex_lock(&cnt_mutex);
12         if (consumed_cnt >= K) {
13             pthread_mutex_unlock(&cnt_mutex);
14             printf("消费者%d: 已完成%d个产品消费, 退出线程\n", consumer_id, K);
15             pthread_exit(NULL);
16         }
17         pthread_mutex_unlock(&cnt_mutex);
18
19         // P操作: 申请满缓冲区
20         sem_wait(&full);
21         // P操作: 申请临界区访问权
22         pthread_mutex_lock(&mutex);
23
24         // 查找满缓冲区
25         BUFFER_ST* full_buf = &buffer[0];
26         while (full_buf->isempty) full_buf = full_buf->nextbuf;
27
28         // 取出产品
29         ITEM_ST consumed_item = full_buf->item;
30         full_buf->isempty = true;
31         consumed_cnt++;
32
33         // 输出行为信息

```

```

34     printf("消费者%d: 准备进入临界区\n", consumer_id);
35     printf("消费者%d: 已进入临界区\n", consumer_id);
36     printf("消费者%d: 从缓冲区%d取出产品%d (生产者: %d, 校验码: %u) \n",
37           consumer_id, full_buf->buffer_id, consumed_item.product_id,
38           consumed_item.producer_id, consumed_item.checksum);
39     printf("消费者%d: 消费产品%d\n", consumer_id, consumed_item.product_id);
40     printf("消费者%d: 已离开临界区\n", consumer_id);
41
42     // V操作: 释放临界区
43     pthread_mutex_unlock(&mutex);
44     // V操作: 增加空缓冲区计数
45     sem_post(&empty);
46 }
47 }

```

(7) 主函数（初始化与线程管理）

代码块

```

1  int main() {
2      pthread_t producers[I], consumers[J];
3      srand((unsigned int)time(NULL)); // 初始化随机数种子
4
5      // 初始化缓冲区
6      init_buffer();
7
8      // 初始化信号量与互斥量
9      sem_init(&empty, 0, N); // 空缓冲区初始值为N
10     sem_init(&full, 0, 0); // 满缓冲区初始值为0
11     pthread_mutex_init(&mutex, NULL);
12     pthread_mutex_init(&cnt_mutex, NULL);
13
14     // 创建生产者线程
15     for (int i = 0; i < I; i++) {
16         int* producer_id = (int*)malloc(sizeof(int));
17         *producer_id = i + 1;
18         pthread_create(&producers[i], NULL, producer_thread,
19             (void*)producer_id);
20     }
21
22     // 创建消费者线程
23     for (int j = 0; j < J; j++) {
24         int* consumer_id = (int*)malloc(sizeof(int));
25         *consumer_id = j + 1;

```

```

25     pthread_create(&consumers[j], NULL, consumer_thread,
    (void*)consumer_id);
26 }
27
28 // 等待所有线程结束
29 for (int i = 0; i < I; i++) pthread_join(producers[i], NULL);
30 for (int j = 0; j < J; j++) pthread_join(consumers[j], NULL);
31
32 // 销毁资源
33 sem_destroy(&empty);
34 sem_destroy(&full);
35 pthread_mutex_destroy(&mutex);
36 pthread_mutex_destroy(&cnt_mutex);
37
38 printf("所有产品生产消费完成, 程序退出\n");
39 return 0;
40 }

```

3. 编译与运行

3. 保存代码为 `producer_consumer.c`。
4. 打开终端，执行编译命令：`gcc producer_consumer.c -o pc -lpthread`（`-lpthread` 链接线程库）。
5. 运行程序：`./pc`，观察终端输出的线程行为与产品信息。

4. 实验验证

- 检查输出是否包含所有要求行为（进入/离开临界区、生产/消费/存入/取出产品）。
- 验证产品编号是否从1到K连续，无重复或缺失。
- 确认生产者生产K个产品后退出，消费者消费K个产品后退出，无忙等待。
- 检查缓冲区操作是否线程安全，无数据错乱（如校验码匹配、生产者/消费者编号对应正确）。

五、实验结果

程序运行后，终端会按时序输出每个生产者和消费者的完整行为。示例输出片段如下：

代码块

```

1  生产者1: 准备进入临界区
2  生产者1: 已进入临界区
3  生产者1: 生产产品1 (校验码: 123456, 生产时间: 1718000000)
4  生产者1: 将产品1存入缓冲区1

```

```
5 生产者1：已离开临界区
6 消费者2：准备进入临界区
7 消费者2：已进入临界区
8 消费者2：从缓冲区1取出产品1（生产者：1，校验码：123456）
9 消费者2：消费产品1
10 消费者2：已离开临界区
11 ...
12 生产者2：已完成20个产品生产，退出线程
13 消费者3：已完成20个产品消费，退出线程
14 所有产品生产消费完成，程序退出
```

所有要求均满足：无超产超消费、无临界区冲突、行为输出完整、边界处理正确。

```
gww@gww-VMware-Virtual-Platform:~/os/第七次$ vim producer_consumer.c
gww@gww-VMware-Virtual-Platform:~/os/第七次$ gcc producer_consumer.c -o pc -lpth
read
gww@gww-VMware-Virtual-Platform:~/os/第七次$ ./pc
生产者2：准备进入临界区
生产者2：已进入临界区
生产者2：生产产品1（校验码：129905，生产时间：1763131899）
生产者2：将产品1存入缓冲区1
生产者2：已离开临界区
消费者2：准备进入临界区
消费者2：已进入临界区
消费者2：从缓冲区1取出产品1（生产者：2，校验码：129905）
消费者2：消费产品1
消费者2：已离开临界区
生产者1：准备进入临界区
生产者1：已进入临界区
生产者1：生产产品2（校验码：132836，生产时间：1763131899）
生产者1：将产品2存入缓冲区1
生产者1：已离开临界区
```

六、实验总结

6. 核心收获：掌握了Posix信号量与互斥量的使用场景，理解了生产者消费者问题中同步（信号量控制缓冲区状态）与互斥（临界区保护）的核心逻辑。
7. 关键难点：通过全局计数器+互斥量解决了线程退出的边界问题，通过循环链表实现缓冲区高效查找，避免了忙等待。
8. 改进方向：可增加图形化输出界面，或通过文件记录输出日志，进一步优化缓冲区查找算法（如记录空/满缓冲区指针，减少遍历开销）。